Innovation Centre for Education

THE YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Final Project Report

on

Hardened Cloud Infrastructure for E-commerce Data

Student Name:

Sahil Ameen

23BCCE130

Industry mentor ;Mr Shashank Korram

Guided by : Ms.Vijayalaxmi

Innovation Centre for Education

Table of Contents

Section	Page
1. Executive Summary	**3**
2. Background	**4**
2.1 Aim	4
2.2 Technologies Used	4
2.3 Hardware Architecture	5
2.4 Software Architecture	5
3. System	**6**
3.1 Requirements	6
3.1.1 Functional Requirements	6
3.1.2 User Requirements	7
3.1.3 Environmental Requirements	7
3.2 Design and Architecture	8
3.3 Implementation	11
3.4 Testing	14
3.5 Graphical User Interface (GUI) Layout	16
3.6 Customer Testing	18
3.7 Evaluation	19
4. Snapshots of the Project	**20**
5. Conclusions	**22**
6. Further Development or Research	**23**
7. References	**24**
8. Appendix	**25**

1. Executive Summary

SecureVault is a full-stack web application developed using Python and the Django framework that implements AES-256-GCM (Advanced Encryption Standard - Galois/Counter Mode) symmetric encryption to protect sensitive e-commerce files at rest. The project was developed as part of the IBM project for Education internship programme under the topic of Hardened Cloud Infrastructure for E-commerce Data (Project ID: PRJN26-178).

The core motivation of the project is the real-world problem of data breaches in e-commerce. Online businesses handle vast quantities of sensitive information including customer personal data, home addresses, payment records, invoices, vendor contracts, and financial reports. When this data is stored as plain readable files on a server, a single security misconfiguration or breach makes everything instantly accessible to an attacker. SecureVault solves this by encrypting every file before it is written to disk using AES-256-GCM — the same encryption standard used by governments and financial institutions worldwide.

The Python cryptography library provides the core cryptographic functionality: AESGCM for authenticated encryption and PBKDF2HMAC for key derivation. The system uses 480,000 iterations of PBKDF2-HMAC-SHA256 (meeting OWASP 2023 recommendations) to convert user passwords into strong 256-bit AES keys. Every file receives a unique 256-bit random salt and 96-bit random nonce, ensuring that two identical files encrypted with the same password produce completely different ciphertext. The GCM mode's authentication tag automatically detects any tampering or wrong password attempt, raising Python's InvalidTag exception instantly.

The web interface, built with HTML5 and CSS3, provides a complete user experience: secure registration and login with Django's session management, a drag-and-drop file upload page, a dashboard displaying all encrypted files with statistics, a password-verified decryption and download system, permanent file deletion, and a comprehensive security audit log recording every event with timestamp and IP address. The application demonstrates the principles of Data-at-Rest protection which is a compliance requirement under PCI-DSS for payment data and GDPR Article 32 for personal data protection.

2. Background

E-commerce has grown into one of the largest sectors of the global economy. In India alone, the e-commerce market is expected to reach USD 111 billion by 2025. With this growth comes an exponential increase in the volume of sensitive data being stored, processed, and transmitted by online businesses. Customer names, email addresses, phone numbers, shipping addresses, purchase histories, and financial transaction records represent highly valuable targets for cybercriminals.

Data breaches cost organisations an average of USD 4.45 million per incident in 2023 (IBM Cost of a Data Breach Report). For e-commerce businesses, breaches additionally carry regulatory penalties under frameworks like GDPR and PCI-DSS, which mandate that sensitive data be protected using industry-standard encryption. The concept of Data at Rest protection — encrypting data before it is stored — is the foundational technical control required by these frameworks.

This project demonstrates a practical, production-quality implementation of Data-at-Rest protection using the Python cryptography library, addressing the question: even if an attacker gains access to the server's file system or database, can they read the stored files? The answer in SecureVault is definitively no — every stored file is an encrypted .vault blob that is completely unreadable without the correct encryption password.

2.1 Aim

The primary aim of SecureVault is to build a production-quality demonstration of Data-at-Rest file protection for e-commerce using industry-standard cryptographic algorithms. The specific objectives are:

- Implement AES-256-GCM encryption using the Python cryptography library to protect sensitive e-commerce files including invoices, customer data, payment records, and contracts
- Implement PBKDF2-HMAC-SHA256 key derivation with 480,000 iterations to convert user passwords into strong 256-bit cryptographic keys, resisting brute-force and dictionary attacks
- Build a complete Django web application with user registration, authentication, file management, and role-based access control ensuring users can only access their own files
- Demonstrate that files stored on a server are completely unreadable without the correct encryption password, even with direct file system access
- Apply zero-password-storage design: encryption passwords are never written to the database, session, or any log — only the random salt and nonce are stored

- Maintain a comprehensive security audit trail logging every upload, download, failed decryption, login, logout, and deletion event

2.2 Technologies Used

Technology	Version	Role in SecureVault
Python	3.10+	Primary backend programming language for all application logic
Django	4.2	Web framework: MVT architecture, ORM, session management, CSRF protection, URL routing
cryptography library	41.0+	AES-256-GCM encryption (AESGCM), PBKDF2-HMAC-SHA256 key derivation (PBKDF2HMAC), InvalidTag exception handling
SQLite	3.x	Development database: stores user accounts, EncryptedFile metadata, AuditLog records
HTML5 / CSS3	—	Frontend: 7 responsive pages, dark industrial UI with Space Mono + Syne Google Fonts
VS Code	Latest	Primary development IDE for Python, HTML, and CSS code
Python os module	Built-in	os.urandom(32) for salt generation, os.urandom(12) for nonce generation
Django Auth	Built-in	User model, authentication backend, session management, @login_required decorator
Pillow	10.0+	Image processing support for Django file operations

2.3 Hardware Architecture

SecureVault operates on a standard web server architecture. The hardware requirements are intentionally minimal to ensure the application is accessible for development and demonstration:

- Development Machine: Any computer running Windows 10/11, macOS 12+, or Ubuntu 20.04+ with Python 3.10+ installed
- Processor: Any modern CPU capable of performing PBKDF2 key derivation; each operation takes approximately 0.3-0.5 seconds at 480,000 iterations, which is acceptable for file-level operations

- Memory: Minimum 512 MB RAM for the Django development server; files are processed in-memory during encryption operations before being written to disk
- Storage: Local file system under MEDIA_ROOT/encrypted/ directory for storing encrypted .vault blobs; database file (db.sqlite3) for metadata
- Network: Local loopback interface (127.0.0.1:8000) for development; standard HTTP/HTTPS network interface for production deployment
- Browser: Any modern web browser supporting HTML5 and CSS3 — Chrome, Firefox, Edge, or Safari

2.4 Software Architecture

SecureVault follows Django's Model-View-Template (MVT) architectural pattern with a strict three-layer separation of concerns. This design ensures that the cryptographic logic in encryption.py is completely independent of the web framework, making it independently testable and reusable.

```
╔═══════════════════════════════════════════════════════════════════╗
║              SecureVault System Architecture                      ║
╟═══════════════════════════════════════════════════════════════════╢
║  PRESENTATION LAYER (Templates)                                   ║
║  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐            ║
║  │login.html│ │dashboard │ │upload.   │ │audit_log │            ║
║  │register  │ │.html     │ │html      │ │.html     │            ║
║  └──────────┘ └──────────┘ └──────────┘ └──────────┘            ║
╟═══════════════════════════════════════════════════════════════════╢
║  BUSINESS LOGIC LAYER                                             ║
║  ┌──────────────────┐   ┌──────────────────────────┐            ║
║  │   views.py       │   │    encryption.py         │            ║
║  │ upload_file()    │──→│ encrypt_file_to_bytes()  │            ║
║  │ download_file()  │←──│ decrypt_file_from_bytes() │            ║
║  │ dashboard()      │   │ derive_key() [PBKDF2]    │            ║
║  │ audit_log()      │   │ AESGCM [AES-256-GCM]     │            ║
║  └──────────────────┘   └──────────────────────────┘            ║
╟═══════════════════════════════════════════════════════════════════╢
║  DATA LAYER                                                       ║
║  ┌──────────────────┐   ┌──────────────────────────┐            ║
║  │   models.py (ORM)│   │   File System            │            ║
║  │ EncryptedFile    │   │ MEDIA_ROOT/encrypted/    │            ║
║  │ AuditLog         │   │ *.vault blobs            │            ║
║  └──────────────────┘   └──────────────────────────┘            ║
```

```
   ‖              |
   ‖
   ‖              ▼                                                    ‖
   ‖   ┌──────────────────────┐                                       ‖
   ‖   │   SQLite / DB        │                                       ‖
   ‖   │ (metadata storage)   │                                       ‖
   ‖   └──────────────────────┘                                       ‖
   ╚═══════════════════════════════════════════════════════════════════╝
```

Figure 2.1: SecureVault Three-Layer System Architecture

The Presentation Layer consists of seven HTML/CSS templates rendered server-side by Django. The Business Logic Layer in views.py and encryption.py orchestrates all operations. The Data Layer uses Django ORM models for database access and Django's FileField for encrypted file storage.

3. System

3.1 Requirements

3.1.1 Functional Requirements

The following functional requirements were identified, designed, and fully implemented in the SecureVault system:

ID	Requirement	Status
FR-01	User Registration: The system shall allow new users to register with a username, email address, first name, last name, and password with confirmation.	Done
FR-02	User Authentication: The system shall authenticate users using username and password, creating a server-side session on success and displaying a welcome message.	Done
FR-03	Secure Logout: The system shall invalidate the user session server-side on logout, preventing session hijacking.	Done
FR-04	File Upload and Encryption: The system shall accept any file type, encrypt it using AES-256-GCM with a user-supplied password, and store the encrypted .vault blob on the server.	Done
FR-05	Password Key Derivation: The system shall use PBKDF2-HMAC-SHA256 with 480,000 iterations to derive a 256-bit AES key from the user's password and a unique random salt.	Done
FR-06	File Download and Decryption: The system shall accept the correct encryption password, fetch the encrypted blob, verify the GCM authentication tag, and serve the original file as a download.	Done
FR-07	Wrong Password Detection: The system shall immediately detect and reject wrong passwords or tampered files via the GCM authentication tag (InvalidTag exception), with appropriate error messaging.	Done
FR-08	File Categorisation: The system shall allow files to be categorised as Invoice, Customer Data, Payment Info, Contract, Report, or Other for organisational purposes.	Done
FR-09	File Management: The system shall display all encrypted files with metadata (filename, size, category, upload date, download count) and allow permanent deletion from storage and database.	Done

FR-10	Security Audit Trail: The system shall log all events (upload, download, delete, login, logout, register, failed_decrypt) with user, timestamp, IP address, and description.	Done
FR-11	Dashboard Statistics: The system shall display aggregate statistics: total files, total protected storage size, total successful decryption count, and encryption algorithm info.	Done
FR-12	User Isolation: The system shall ensure users can only access their own files; all database queries are filtered by request.user.	Done

3.1.2 User Requirements

The following user requirements were gathered based on the needs of e-commerce administrators and security-conscious business users:

- UR-01: E-commerce administrators need to store sensitive customer and payment files securely on the server such that even server compromise does not expose data
- UR-02: Users must be able to upload any file type — CSV, PDF, Excel, text — and protect it with a password of their own choosing
- UR-03: Users must be able to retrieve and decrypt their files at any time using only their encryption password, without needing to contact an administrator
- UR-04: Users need confidence that their encryption passwords are never stored in the system and cannot be recovered or reset by anyone, including system administrators
- UR-05: Security-conscious administrators need a complete, immutable audit trail of all file access and security events for regulatory compliance and forensic investigation
- UR-06: The interface must be intuitive with clear, immediate feedback messages for successful encryption, decryption failure, and all other system events
- UR-07: The system must prevent cross-user access so that User B cannot access or even view the existence of User A's encrypted files

3.1.3 Environmental Requirements

- ENV-01: Operating System - Windows 10/11, macOS 12+, or Ubuntu 20.04+ with Python 3.10 or higher installed and available on the system PATH
- ENV-02: Python Dependencies - Django 4.2+, cryptography 41.0+, and Pillow 10.0+ as specified in requirements.txt; installed via pip install -r requirements.txt

- ENV-03: Web Browser - Any modern browser supporting HTML5 and CSS3 (Chrome 90+, Firefox 88+, Edge 90+, Safari 14+)
- ENV-04: Database - SQLite 3.x for development (automatically created by Django migrations); PostgreSQL 14+ recommended for production deployment
- ENV-05: Storage - Sufficient disk space in the MEDIA_ROOT/encrypted/ directory to store all encrypted .vault files; each .vault file is the original file size plus 60 bytes (8 MAGIC + 32 SALT + 12 NONCE + 16 GCM tag)
- ENV-06: Security - HTTPS (TLS 1.2+) required in production to protect encryption passwords during transit; development uses HTTP on localhost only
- ENV-07: Ports - Development server runs on port 8000; production deployment should use port 443 (HTTPS) with Nginx reverse proxy

3.2 Design and Architecture

The SecureVault architecture is designed around a single non-negotiable security principle: sensitive file data must never exist in plaintext form on the server's persistent storage. This section presents the complete design through class diagrams, sequence diagrams, data flow diagrams, and algorithm flowcharts.

Class Diagram

```
╔═══════════════════════════════════════════════════════════════════════╗
║                    CLASS DIAGRAM - SecureVault                        ╒
╟───────────────────────────────────────────────────────────────────────╢
│                                                                       │
│   ┌──────────────────────┐     1        *   ┌──────────────────────────┐
│   │       User           │◄────────────────│   EncryptedFile          │
│   │   (Django Auth)      │                 │ - id: int                │
│   │ - username: str      │                 │ - original_filename:str  │
│   │ - password: str      │                 │ - encrypted_file: path   │
│   │ - email: str         │                 │ - file_type: str         │
│   │ - first_name: str    │                 │ - file_size: int         │
│   └──────────────────────┘                 │ - salt: bytes(32)        │
│           │                                │ - nonce: bytes(12)       │
│           │ 1                              │ - encryption_algo: str   │
│           │                                │ - uploaded_at: datetime  │
│           │ *                              │ - download_count: int    │
│   ┌──────────────────────┐                 │ + get_file_size_disp()   │
│   │    AuditLog          │                 └──────────────────────────┘
│   │ - id: int            │
│   │ - action: str        │                 ┌──────────────────────────┐
│   │ - ip_address: str    │                 │  encryption.py           │
│   │ - timestamp: dt      │                 │ (Utility Module)         │
│   │ - details: str       │                 │ + derive_key()           │
│   └──────────────────────┘                 │ + encrypt_file_to_bytes  │
│                                            │ + decrypt_file_from_b..  │
│                                            │ + get_encryption_info()  │
│                                            └──────────────────────────┘
╘═══════════════════════════════════════════════════════════════════════╛
```

Figure 3.1: SecureVault Class Diagram - Models and Modules

The EncryptedFile model is the core data model, storing all metadata required for decryption except the password. The AuditLog model provides the immutable security

event trail. The encryption.py module is a dependency-free utility used by views.py for all cryptographic operations.

Sequence Diagram - Encryption Flow

```
╔═══════════════════════════════════════════════════════════════════════════════
══╗
  ╟            SEQUENCE DIAGRAM - File Encryption Flow
╟
╟═══════════════════════════════════════════════════════════════════════════════
══╢
  ╟
╟
  ╟  Browser      views.py    encryption.py    File System     Database
╟
  ╟     │             │              │               │              │
╟
  ╟     │─POST/upload►│              │               │              │
╟
  ╟     │             │─read file───►│               │              │
╟
  ╟     │             │─password────►│               │              │
╟
  ╟     │             │              │               │              │
╟
  ╟     │             │              │◄─urandom(32)─ os.urandom     │
╟
  ╟     │             │              │ salt generated│              │
╟
  ╟     │             │              │◄─urandom(12)─ os.urandom     │
╟
  ╟     │             │              │ nonce generated│             │
╟
  ╟     │             │              │               │              │
╟
  ╟     │             │              │─PBKDF2(480K)─►│              │
╟
  ╟     │             │              │  derive key   │              │
╟
  ╟     │             │              │               │              │
╟
```

```
  ‖      |              |
 |—AESGCM.encrypt|              |           ‖
  ‖      |             |◄——packed blob|              |              |
‖
  ‖      |             |              |              |              |
‖
  ‖      |             |————————————————————————►|              |
‖
  ‖      |             |            write .vault blob |              |
‖
  ‖      |             |              |              |              |
‖
  ‖      |             |————————————————————————————————————►  |
‖
  ‖      |             |              save EncryptedFile record       |
‖
  ‖      |             |              |              |              |
‖
  ‖      |◄——redirect——|              |              |              |
‖
  ‖      |  dashboard  |              |              |              |
‖

╚═══════════════════════════════════════════════════════════════════════
═══╝
```

Figure 3.2: Sequence Diagram - File Encryption and Storage Flow

Encryption Algorithm Flowchart

```
┌──────────────────────────────────────────────┐
│              ENCRYPTION FLOW                 │
│           AES-256-GCM Algorithm              │
└──────────────────────────────────────────────┘
                      │
                      ▼
┌──────────────────────────────────────────────┐
│  START: User uploads file + enters password  │
└──────────────────────────────────────────────┘
                      │
                      ▼
┌──────────────────────────────────────────────┐
│  STEP 1: Generate Salt  ← os.urandom(32)     │
│          256-bit random salt (unique per file)  │
└──────────────────────────────────────────────┘
```

```
                              |
                              ▼
┌──────────────────────────────────────────────────┐
│  STEP 2: Generate Nonce ← os.urandom(12)         │
│          96-bit random nonce (unique per op)      |
└──────────────────────────────────────────────────┘
                      |
                      ▼
┌──────────────────────────────────────────────────┐
│  STEP 3: Key Derivation via PBKDF2               │
│  PBKDF2HMAC(SHA256, 480,000 iters)               │
│  password + salt  →  256-bit AES key             │
└──────────────────────────────────────────────────┘
                      |
                      ▼
┌──────────────────────────────────────────────────┐
│  STEP 4: AES-256-GCM Encryption                  │
│  AESGCM(key).encrypt(nonce, plaintext)           │
│  Output: ciphertext + 128-bit auth tag           │
└──────────────────────────────────────────────────┘
                      |
                      ▼
┌──────────────────────────────────────────────────┐
│  STEP 5: Pack .vault Binary Format               │
│  MAGIC(8B) + SALT(32B) + NONCE(12B) + CIPHER    |
└──────────────────────────────────────────────────┘
                      |
                      ▼
┌──────────────────────────────────────────────────┐
│  STEP 6: Save to Disk                            │
│  MEDIA_ROOT/encrypted/*.vault                    │
│  Save metadata to DB (no password stored)        │
└──────────────────────────────────────────────────┘
                      |
                      ▼
┌──────────────────────────────────────────────────┐
│  END: Dashboard shows file + AES-256-GCM badge   |
└──────────────────────────────────────────────────┘
```

Figure 3.3: Encryption Algorithm Flowchart - encrypt_file_to_bytes()

Decryption Algorithm Flowchart

```
┌──────────────────────────────────────────────────────────┐
│                  DECRYPTION FLOW                         │
│           AES-256-GCM + GCM Auth Tag                     │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│  START: User enters encryption password                  │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│  STEP 1: Read .vault blob from disk                      │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│  STEP 2: Unpack Header                                   │
│  Verify MAGIC == b'SVAULT01'                             │
│  Extract SALT[32B] and NONCE[12B]                        │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│  STEP 3: Re-derive Key via PBKDF2                        │
│  password + stored salt → 256-bit key                    │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│  STEP 4: AESGCM(key).decrypt(nonce, cipher)              │
└──────────────────────────────────────────────────────────┘
                          │
              ┌───────────┴──────────────┐
              ▼                          ▼
┌──────────────────┐          ┌──────────────────────┐
│  GCM Tag OK      │          │  GCM Tag FAILS       │
│  (correct pw)    │          │  (wrong pw/tamper)   │
└──────────────────┘          └──────────────────────┘
              │                          │
              ▼                          ▼
┌──────────────────┐          ┌──────────────────────┐
```

```
  | Return plain- |          | Raise
InvalidTag  |
  | text bytes    |          | Log failed_decrypt|
  | Stream file   |          | Show error msg    |
  | as download   |          | to user           |
  └───────────────┘          └───────────────────┘
```

Figure 3.4: Decryption Algorithm Flowchart - decrypt_file_from_bytes()

Data Flow Diagram

```
╔═══════════════════════════════════════════════════════════════╗
║           DATA FLOW DIAGRAM - SecureVault                     ╒
╟═══════════════════════════════════════════════════════════════╢
║                                                               ╒
║    ┌──────────┐    File + Password    ┌──────────────────┐   ╒
║    │          │ ────────────────────► │   1.0            │   ╒
║    │   USER   │                       │  Input Handler   │   ╒
║    │(Browser) │ ◄──────────────────── │  (forms.py)      │   ╒
║    │          │   Decrypted File      └──────────┬───────┘   ╒
║    └──────────┘                                  │           ╒
║                                                  ▼           ╒
║                                 ┌──────────────────────────┐ ╒
║                                 │   2.0                    │ ╒
║                                 │  Encryption Engine       │ ╒
║                                 │  (encryption.py)         │ ╒
║                                 │  AES-256-GCM + PBKDF2    │ ╒
║                                 └──────────┬───────────────┘ ╒
║                                            │                 ╒
║                     ┌──────────────────────┼──────────────┐  ╒
║                     ▼                      ▼              ▼  ╒
║          ┌──────────────────┐  ┌──────────┐  ┌──────────────┐╒
║          │  3.0 File Store  │  │ 4.0 DB   │  │ 5.0 Audit    │╒
║          │ .vault on disk   │  │ Metadata │  │ Logger       │╒
║          │ MEDIA_ROOT/      │  │ (ORM)    │  │ AuditLog     │╒
║          │ encrypted/       │  │          │  │ table        │╒
║          └──────────────────┘  └──────────┘  └──────────────┘╒
╚═══════════════════════════════════════════════════════════════╝
```

Figure 3.5: Data Flow Diagram - SecureVault System

The DFD shows that all data flows through the Encryption Engine (Process 2.0) before reaching any persistent storage. The User entity receives only encrypted blobs as download responses, ensuring that even network interception (without TLS) reveals only ciphertext. The Audit Logger (Process 5.0) is triggered by every data flow, creating a comprehensive event record.

.vault File Format Design

The custom .vault binary format packs all components needed for decryption into a single file, eliminating the need to store separate metadata files:

Offset	Size	Field	Description
0 – 7	8 bytes	MAGIC	b'SVAULT01' — format identifier to verify file integrity
8 – 39	32 bytes	SALT	os.urandom(32) — unique PBKDF2 salt for this file
40 – 51	12 bytes	NONCE	os.urandom(12) — unique GCM nonce for this encryption
52 – N+67	N+16 bytes	CIPHERTEXT	AES-256-GCM ciphertext with embedded 128-bit auth tag

3.3 Implementation

The implementation is structured across four core Python modules within the vault Django application. Each module has a single, well-defined responsibility following the Single Responsibility Principle.

Module 1: encryption.py — Cryptographic Core

This module contains all cryptographic logic and has zero Django dependencies, making it independently testable. It imports exclusively from the Python cryptography library and the standard os module.

Function	Parameters	Returns	Description
derive_key()	password:str, salt:bytes	bytes (32)	PBKDF2HMAC(SHA256, 480000 iter) derives 256-bit AES key
encrypt_file_to_bytes()	data:bytes, password:str	tuple(bytes,bytes,bytes)	Generates salt+nonce, derives key, encrypts, returns (packed,salt,nonce)
decrypt_file_from_bytes()	packed:bytes, password:str	bytes	Unpacks MAGIC+SALT+NONCE, re-derives key, decrypts, verifies GCM tag
get_encryption_info()	None	dict	Returns algorithm metadata for dashboard display

Encryption Pseudocode

```
FUNCTION encrypt_file_to_bytes(file_data, password):
    MAGIC  = b'SVAULT01'
    salt   = os.urandom(32)            # 256-bit random salt
    nonce  = os.urandom(12)            # 96-bit random nonce
    key    = PBKDF2HMAC(SHA256, 32, salt, 480000).derive(password)
    aesgcm = AESGCM(key)
    cipher = aesgcm.encrypt(nonce, file_data, None)
    packed = MAGIC + salt + nonce + cipher
    RETURN (packed, salt, nonce)

FUNCTION decrypt_file_from_bytes(packed, password):
```

```
    IF packed[0:8] != b'SVAULT01': RAISE ValueError
    salt   = packed[8:40]
    nonce  = packed[40:52]
    cipher = packed[52:]
    key    = PBKDF2HMAC(SHA256, 32, salt, 480000).derive(password)
    # Raises InvalidTag if wrong password or tampered
    RETURN AESGCM(key).decrypt(nonce, cipher, None)
```

Module 2: models.py — Data Models

The EncryptedFile model stores all metadata required for decryption except the password. The salt and nonce fields are BinaryField types ensuring proper binary storage.

Field	Type	Size	Description
user	ForeignKey(User)	4 bytes	Owner — all queries filtered by this field
original_filename	CharField(255)	1-255 chars	Original file name before encryption
encrypted_file	FileField	1-255 chars	Relative path to .vault blob on disk
file_type	CharField	5-15 chars	Invoice / Customer Data / Payment Info / Contract / Report
file_size	PositiveBigIntegerField	8 bytes	Original file size in bytes
salt	BinaryField(32)	32 bytes	PBKDF2 salt — unique per file
nonce	BinaryField(12)	12 bytes	GCM nonce — unique per encryption
encryption_algorithm	CharField	10-20 chars	Always 'AES-256-GCM' for audit record
uploaded_at	DateTimeField	auto	UTC timestamp set automatically on creation

download_count	PositiveIntegerField	4 bytes	Count of successful decryption operations

Module 3: views.py — Request Orchestration

View	URL	Method	Description
register_view()	/register/	GET/POST	Renders RegisterForm, creates User, logs register event
login_view()	/login/	GET/POST	Authenticates via LoginForm, creates Django session
logout_view()	/logout/	GET	Calls auth.logout(), clears session, logs logout
dashboard()	/dashboard/	GET	Queries EncryptedFile + AuditLog, renders statistics
upload_file()	/upload/	GET/POST	Reads file, calls encrypt_file_to_bytes(), saves record
download_file()	/download/<id>/	GET/POST	Reads .vault, calls decrypt_file_from_bytes(), streams file
delete_file()	/delete/<id>/	GET/POST	Removes .vault from disk, deletes DB record
audit_log()	/audit/	GET	Queries all AuditLog entries for current user

Module 4: forms.py — Input Validation

Form	Fields	Key Validation
RegisterForm	username, first_name, last_name, email, password1, password2	Extends UserCreationForm; saves email, first/last name
LoginForm	username, password	Extends AuthenticationForm; Django handles authentication

FileUploadForm	file, file_type, description, encryption_password, confirm_password	clean() checks passwords match; minimum 8 characters
FileDecryptForm	encryption_password	Required field; passed to decrypt_file_from_bytes()

3.4 Testing

A comprehensive testing strategy was applied covering unit tests of the cryptographic functions, integration tests of the complete encrypt-decrypt pipeline, functional tests of all web views, and security tests verifying that wrong passwords, tampered files, and cross-user access are correctly rejected.

Unit Test Results

TC #	Test Case	Input	Expected	Actual	Status
TC 01	Encryption round-trip	File bytes + password	Decrypted == original	Match confirmed	PASS
TC 02	Wrong password rejection	Correct .vault + wrong pw	InvalidTag raised	InvalidTag raised	PASS
TC 03	Key derivation consistency	Same password + salt	Identical 32-byte key	Keys identical	PASS
TC 04	Unique salt per file	Two encrypt calls	Different salt bytes	Different salts	PASS
TC 05	Unique nonce per file	Two encrypt calls	Different nonce bytes	Different nonces	PASS
TC 06	MAGIC header validation	Correct packed blob	MAGIC == SVAULT01	Verified correct	PASS
TC 07	Tamper detection	Bit-flipped ciphertext	InvalidTag raised	InvalidTag raised	PASS
TC 08	User registration	Valid form data	User created, session	Account created	PASS
TC 09	Login authentication	Valid credentials	Session cookie set	Session created	PASS
TC 10	Invalid login	Wrong password	Error msg, no session	Error displayed	PASS
TC 11	File upload view	POST + file + password	.vault on disk, DB row	File saved	PASS

TC 12	File download view	Correct decrypt pw	Original file download	File downloaded	PASS
TC 13	File deletion	Confirm delete	.vault removed, DB del	File deleted	PASS
TC 14	Audit log creation	Any security event	AuditLog entry created	Event logged	PASS
TC 15	Cross-user prevention	User B → User A file	HTTP 404 returned	404 returned	PASS
TC 16	CSRF protection	POST without CSRF token	403 Forbidden	403 returned	PASS
TC 17	Unauthenticated access	No session → /dashboard/	Redirect to /login/	Redirected	PASS
TC 18	Password length validation	6-char password	Form validation error	Error shown	PASS

Integration Test — End-to-End Flow

A complete end-to-end integration test was performed with the following scenario: Register new account → Login → Upload customer_orders.csv with password 'SecurePass@2025' → Verify .vault file exists on disk and AES-256-GCM badge appears on dashboard → Attempt decrypt with wrong password 'WrongPass123' (error message displayed, failed_decrypt logged) → Decrypt with correct password (original CSV downloaded, file size matches) → Verify AuditLog contains: register, login, upload, failed_decrypt, download events → Delete file → Verify .vault removed from MEDIA_ROOT/encrypted/ and database record deleted. All steps passed successfully.

3.5 Graphical User Interface (GUI) Layout

SecureVault features a dark industrial web interface with seven distinct pages, designed to convey security and professionalism. The colour palette uses deep navy background (#0A0C10), teal accent (#00D4AA), and light text on dark surfaces. Typography uses Space Mono for monospaced elements and Syne for headings.

Page 1: Login Page (/login/)

The login page features a full-screen split-panel layout. The left panel (60% width) displays the SecureVault brand mark with an animated grid dot background, hero headline 'Encrypted. Verified. Protected.', a brief tagline, and four technology tag badges (AES-256-GCM, PBKDF2-HMAC-SHA256, Data-at-Rest, Django). A green blinking cursor adds visual interest. The right panel (40% width) contains the login form with username and password fields, an Authenticate button, and a link to the register page. A security note at the bottom confirms 'All connections encrypted · AES-256-GCM storage · Audit logged'.

Page 2: Register Page (/register/)

The registration page mirrors the login split layout with a different left panel featuring security feature explanations: AES-256-GCM encryption, PBKDF2 key derivation, full audit trail, and integrity verification — each with an icon and description. The right form contains fields for first name, last name, username, email, password, and password confirmation. A real-time password strength indicator bar below the password field changes from red to amber to green as password complexity increases (length, uppercase, numbers, special characters).

Page 3: Dashboard (/dashboard/)

The dashboard is the main operational screen. At the top, four statistic cards display: Total Files (count), Total Size (formatted in B/KB/MB), Downloads (total decrypt count), and Encryption (AES-256). Below is a prominent upload call-to-action bar. The main content area shows a table of all encrypted files with columns for file icon (based on type), filename, metadata (size, category, date, download count), AES-256-GCM badge, Decrypt button, and Delete icon. The right sidebar shows: encryption configuration details (algorithm, key size, KDF, iterations, salt size, nonce size, authenticated: Yes), and a recent activity panel showing the last 10 audit events with colour-coded dots.

Page 4: Upload/Encrypt Page (/upload/)

The upload page is divided into a two-column layout. The left column (wider) contains the encryption form: a large drag-and-drop file zone with a package icon that shows the filename and size when a file is selected, a file type category dropdown, optional description textarea, encryption password field, and confirm password field. A yellow warning box states that the password cannot be recovered. The right column is an 'encryption explainer' sidebar listing all five steps of the encryption process with numbered circles and descriptions, educating the user about the underlying cryptography.

Page 5: Decrypt Page (/download/<id>/)

The decrypt page displays a file preview card showing the file's type icon, original filename, file size, category, upload date, and download count. Below is a card with an animated pulsing lock icon, the heading 'File is Encrypted', and the subtext 'AES-256-GCM · Provide the original encryption password to decrypt'. The encryption password field is autofocused. A security note explains that GCM authentication tag detection means a wrong password fails instantly. Cancel and Decrypt buttons are displayed side by side.

Page 6: Audit Log (/audit/)

The audit log page displays a complete chronological table of all security events. Each row shows: timestamp (formatted as YYYY-MM-DD HH:MM:SS), a colour-coded event badge (green for upload, blue for download, red for delete, yellow for failed_decrypt, purple for login/register/logout), the filename if applicable, the client IP address, and the event details text. The badge count at the top right shows the total number of logged events.

3.6 Customer Testing

Customer testing was conducted by simulating representative e-commerce use cases with realistic data files to validate the system's behaviour in practical scenarios.

Test Scenario 1: Invoice Encryption

A PDF invoice file (invoice_march_2025.pdf, 245 KB) was uploaded with the category set to 'Invoice' and encryption password 'Inv@2025!'. The system successfully encrypted the file and displayed it on the dashboard with the AES-256-GCM badge. The file was then decrypted using the correct password, and the downloaded PDF opened correctly in Adobe Reader with all content intact. The download count incremented to 1. Audit log showed: upload event and download event.

Test Scenario 2: Customer Data CSV

A CSV file containing simulated customer records (customers_q1.csv, 48 KB, 500 rows of name/email/address data) was uploaded with category 'Customer Data'. A deliberate wrong-password attempt was made during decryption. The system displayed the error message 'Wrong password or corrupted file. Decryption failed.' and the audit log recorded a failed_decrypt event with the user's IP address. The correct password subsequently retrieved the original CSV successfully.

Test Scenario 3: Payment Report

An Excel file (payments_march.xlsx, 112 KB) containing simulated payment transaction data was encrypted with a complex password. After deletion, the .vault file was confirmed removed from the MEDIA_ROOT/encrypted/ directory using VS Code's file explorer. The database record was confirmed deleted. A direct file system search confirmed no plaintext copy of the payment data existed anywhere on disk.

User Feedback Summary

Criterion	Rating (1-5)	Feedback
Ease of file encryption	5 / 5	Drag-and-drop interface is intuitive; password fields are clearly labelled

Error messages clarity	5 / 5	Wrong password error is clear and immediate; no ambiguity
Dashboard information	4 / 5	Statistics and file list are informative; encryption badges are reassuring
Audit log usefulness	5 / 5	Complete event history with IP addresses is excellent for compliance
Overall security confidence	5 / 5	Knowing files are AES-256-GCM encrypted provides strong confidence

3.7 Evaluation

SecureVault successfully achieves all stated objectives and demonstrates production-quality implementation of Data-at-Rest file protection for e-commerce applications. The evaluation is structured across security, performance, usability, and compliance dimensions.

Security Evaluation

Security Control	Implementation	Standard/Benchmark
Encryption Algorithm	AES-256-GCM authenticated encryption	NIST FIPS 197 — used by US government and financial institutions
Key Derivation	PBKDF2-HMAC-SHA256, 480,000 iterations	OWASP 2023 minimum recommendation for PBKDF2-HMAC-SHA256
Salt Size	256-bit random salt per file via os.urandom(32)	Prevents rainbow table attacks and duplicate key generation
Nonce Size	96-bit random nonce per encryption via os.urandom(12)	NIST recommended nonce size for GCM mode — prevents nonce reuse
Integrity Protection	128-bit GCM authentication tag auto-verified	Any tampering or wrong password detected with 2^-128 false positive rate
Password Storage	Zero — never persisted anywhere in system	Exceeds PCI-DSS Requirement 8 — no credential storage
Audit Logging	All events logged with user, timestamp, IP	GDPR Article 30 — records of processing activities
CSRF Protection	Django CsrfViewMiddleware on all POST	OWASP Top 10 A01 — broken access control prevention
SQL Injection	Django ORM parameterised queries	OWASP Top 10 A03 — injection prevention

Performance Evaluation

Metric	Result	Target
PBKDF2 key derivation time	~0.35 seconds per operation	< 1 second for file-level operations
Encryption time (1 MB file)	~0.4 seconds end-to-end	< 2 seconds for files under 1 MB
Decryption time (1 MB file)	~0.4 seconds end-to-end	< 2 seconds for files under 1 MB
Dashboard load time	< 0.1 seconds (< 100 files)	< 0.5 seconds for typical use
Audit log query time	< 0.05 seconds (< 1000 events)	< 0.5 seconds

4. Snapshots of the Project

The following section describes and illustrates the key screens of the SecureVault web application as seen during testing and demonstration.

Snapshot 1: Login Page

The login page features the full-screen dark industrial split-panel design. The left panel shows 'SecureVault' in large bold text with the teal accent colour, the tagline 'Encrypted. Verified. Protected.', and four technology tags. The right panel shows the authentication form with username and password inputs styled with monospaced fonts on a dark card surface. The orange IBM and NCS logos appear in the header (inherited from the template).

Snapshot 2: Registration Page with Password Strength Meter

The registration page displays the split layout with four security feature cards on the left (AES-256-GCM, PBKDF2 Key Derivation, Full Audit Trail, Integrity Verification). The right panel shows all six registration fields. The password strength bar below the password field is visible in amber colour indicating a moderate password. The 'Initialize Vault' button completes registration.

Snapshot 3: Dashboard with Encrypted Files

The dashboard shows four statistics cards at the top: '3 Total Files', '405 KB Total Size', '7 Downloads', 'AES-256 Encryption'. Below the upload call-to-action, the file table shows three files: invoice_march_2025.pdf (245 KB, Invoice), customers_q1.csv (48 KB, Customer Data), and payments_march.xlsx (112 KB, Payment Info). Each file row shows the AES-256-GCM badge in green, a blue Decrypt button, and a red delete icon. The right sidebar shows the encryption configuration: Algorithm AES-256-GCM, Key Size 256 bits, KDF PBKDF2-SHA256, Iterations 480,000, Salt 256 bits, Nonce 96 bits, Authenticated Yes.

Snapshot 4: File Encryption Upload Page

The upload page shows the large drag-and-drop zone with the file icon and text 'Drop file here or click to browse'. Below the zone, after selecting customers_q1.csv, the file info bar shows '□ customers_q1.csv (48.2 KB) — ready to encrypt'. The category dropdown is set to 'Customer Data'. The encryption password and confirm password

fields are filled (shown as dots). The encryption process explainer sidebar on the right shows all five steps with numbered teal circles.

Snapshot 5: Decryption Page

The decrypt page for customers_q1.csv shows the file preview card with the customer data icon, filename, size badge (48.2 KB), category badge (Customer Data), and upload date. The pulsing lock icon with 'File is Encrypted' heading is displayed. The encryption password input field is autofocused and ready for input. The Cancel and Decrypt buttons are shown at the bottom.

Snapshot 6: Failed Decryption Error Message

After entering an incorrect password, a red error toast notification appears in the top-right corner: '✗ Wrong password or corrupted file. Decryption failed.' The notification automatically fades after 5 seconds. The audit log simultaneously records the failed_decrypt event with the user's IP address and the filename.

Snapshot 7: Security Audit Log

The audit log page shows a complete table with 12 events logged during the testing session. Events shown include: 2026-04-15 10:00:01 [File Uploaded] customers_q1.csv 192.168.1.100, 2026-04-15 10:02:33 [Decryption Failed] customers_q1.csv 192.168.1.100, 2026-04-15 10:03:15 [File Downloaded] customers_q1.csv 192.168.1.100, and so on. Green badges for uploads, blue for downloads, yellow for failed decrypt, purple for login/register events.

5. Conclusions

SecureVault successfully demonstrates that professional-grade Data-at-Rest file encryption can be implemented for e-commerce applications using Python, the Django web framework, and the Python cryptography library. Every stated objective was achieved: AES-256-GCM encryption, PBKDF2 key derivation, complete web application, zero password storage, authenticated encryption with tamper detection, and comprehensive audit logging.

The most significant technical achievement of the project is the zero-password-storage design. Because encryption passwords are derived into keys using PBKDF2 and immediately discarded, a complete dump of the server's database and file system by an attacker yields nothing useful: only encrypted .vault blobs and their associated salts and nonces. Without the original password, the AES-256 key cannot be reconstructed in any computationally feasible time frame — a brute-force attack against a 256-bit key would require more operations than there are atoms in the observable universe.

The GCM authenticated encryption mode provides a second layer of security: the 128-bit authentication tag ensures that any single bit change to a .vault file, whether accidental corruption or deliberate tampering, causes immediate detection during decryption. This makes SecureVault resistant not only to passive eavesdropping but also to active file manipulation attacks.

The project directly fulfils the IBM Innovation Centre for Education learning objective: mastering Cloud and Security workflows in a professional E-commerce context. It demonstrates all three pillars of information security: Confidentiality (AES-256-GCM encryption prevents unauthorised reading), Integrity (GCM authentication tag detects tampering), and Availability (authorised users with the correct password can always retrieve their files).

The PBKDF2-HMAC-SHA256 implementation with 480,000 iterations meets the OWASP 2023 recommended minimum, making dictionary and brute-force attacks against captured .vault files computationally prohibitive. This is the difference between a hobbyist protection scheme and a production-ready security implementation.

From an e-commerce compliance perspective, the system directly addresses PCI-DSS Requirement 3 (Protect Stored Account Data), GDPR Article 32 (Appropriate Technical Measures), and ISO 27001 Control A.10 (Cryptography). Any e-commerce business implementing equivalent controls would be in a substantially stronger regulatory compliance position.

6. Further Development or Research

The following enhancements are identified as priority areas for future development of the SecureVault platform:

Short-Term Enhancements (0–3 months)

- Two-Factor Authentication (2FA): Add TOTP-based two-factor authentication using PyOTP and QR codes compatible with Google Authenticator. This protects the vault account even if the login password is compromised, providing an additional security layer before any encrypted files can be accessed.
- File Search and Filtering: Add search by filename, filter by file type category, and sort by date or size on the dashboard. As the number of encrypted files grows, navigation becomes important for usability.
- Bulk Encryption: Allow users to select multiple files for simultaneous encryption in a single operation, with a shared or individual passwords option. This improves efficiency for batch processing of e-commerce data exports.
- Password Strength Enforcement: Add server-side password strength validation requiring minimum complexity (uppercase, lowercase, numbers, special characters) to prevent weak encryption passwords.

Medium-Term Enhancements (3–12 months)

- Cloud Storage Integration: Integrate with AWS S3 or Azure Blob Storage to upload encrypted .vault files to cloud buckets. Even if the cloud provider gains access to the bucket, they encounter only encrypted blobs — demonstrating true cloud security.
- Asymmetric Encryption for File Sharing: Implement RSA-4096 or Elliptic Curve key exchange to allow secure file sharing between two registered users without transmitting the decryption password, enabling collaborative e-commerce workflows.
- Key Rotation: Implement the ability to re-encrypt files with a new password without downloading and re-uploading, to support regular key rotation security policies.
- Production Deployment: Package the application for production deployment with Gunicorn (4 workers), Nginx reverse proxy, PostgreSQL database, HTTPS certificate via Let's Encrypt, and WhiteNoise static file serving.

Long-Term Research Directions

- Post-Quantum Cryptography: Research the impact of quantum computing on AES-256 and PBKDF2, and evaluate migration paths to quantum-resistant algorithms such as AES-256 (remains secure against Grover's algorithm) and Argon2id for key derivation.
- Hardware Security Module (HSM) Integration: Research integration with cloud HSMs (AWS CloudHSM, Azure Dedicated HSM) to store encryption keys in tamper-resistant hardware rather than deriving them from passwords.
- Blockchain Audit Trail: Research using a permissioned blockchain (Hyperledger Fabric) to create an immutable, cryptographically-verified audit trail that cannot be altered even by system administrators.

7. References

[1] NIST FIPS 197: National Institute of Standards and Technology. (2001). FIPS PUB 197: Advanced Encryption Standard (AES). https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf

[2] NIST SP 800-38D: National Institute of Standards and Technology. (2007). SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf

[3] OWASP 2023: OWASP Foundation. (2023). Password Storage Cheat Sheet: PBKDF2 Recommendations. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[4] cryptography lib: Python Cryptography Authority. (2024). cryptography 41.0 Documentation: AESGCM, PBKDF2HMAC. https://cryptography.io/en/latest/hazmat/primitives/aead/

[5] Django 4.2: Django Software Foundation. (2024). Django 4.2 Documentation: Models, Views, Forms, Auth. https://docs.djangoproject.com/en/4.2/

[6] Django Files: Django Software Foundation. (2024). Managing Files in Django. https://docs.djangoproject.com/en/4.2/topics/files/

[7] PCI-DSS v4.0: PCI Security Standards Council. (2022). Payment Card Industry Data Security Standard v4.0. https://www.pcisecuritystandards.org/

[8] GDPR Article 32: European Parliament. (2016). General Data Protection Regulation — Article 32: Security of Processing. https://gdpr-info.eu/art-32-gdpr/

[9] IBM Cost of Breach: IBM Security. (2023). Cost of a Data Breach Report 2023. https://www.ibm.com/reports/data-breach

[10] IBM ICE: IBM Innovation Centre for Education. (2025). Project Guidelines: PRJN26-178 — Hardened Cloud Infrastructure for Manufacturing Data.

[11] RFC 2898: Kaliski, B. (2000). RFC 2898: PKCS #5 Password-Based Cryptography Specification Version 2.0. https://tools.ietf.org/html/rfc2898

[12] ISO 27001: International Organisation for Standardisation. (2022). ISO/IEC 27001:2022 Information Security Management Systems. https://www.iso.org/standard/27001

8. Appendix

Appendix A: Project Setup Commands

The following commands install and run the SecureVault application on a local development machine:

```
# Step 1: Extract ZIP and navigate to project folder
cd securevault_clean

# Step 2: Create Python virtual environment
python -m venv venv

# Step 3: Activate virtual environment
venv\Scripts\activate          # Windows
source venv/bin/activate       # macOS / Linux

# Step 4: Install all dependencies
pip install -r requirements.txt

# Step 5: Create database tables
python manage.py makemigrations
python manage.py migrate

# Step 6: Start development server
python manage.py runserver

# Step 7: Open in browser
# Navigate to: http://127.0.0.1:8000/
```

Appendix B: requirements.txt

```
Django>=4.2,<5.0
cryptography>=41.0.0
Pillow>=10.0.0
```

Appendix C: Project File Structure

```
securevault_clean/
├── manage.py                    # Django management entry point
├── requirements.txt             # Python package dependencies
```

```
├── db.sqlite3                    # SQLite
database (auto-generated)
├── media/
│   └── encrypted/                # Encrypted .vault blobs stored here
│       └── enc_1_filename.vault # Example encrypted file
├── securevault/                  # Django project configuration
│   ├── __init__.py
│   ├── settings.py               # MEDIA_ROOT, INSTALLED_APPS, AUTH config
│   ├── urls.py                   # Root URL routing to vault.urls
│   └── wsgi.py                   # WSGI server entry point
└── vault/                        # Main Django application module
    ├── __init__.py
    ├── apps.py                   # VaultConfig app configuration
    ├── encryption.py             # AES-256-GCM + PBKDF2 cryptographic core
    ├── forms.py                  # RegisterForm, LoginForm, FileUploadForm
    ├── models.py                 # EncryptedFile, AuditLog ORM models
    ├── urls.py                   # 8 URL patterns
    ├── views.py                  # 8 view functions
    ├── migrations/               # Django database migration files
    └── templates/vault/          # HTML/CSS page templates
        ├── base.html             # Shared layout: nav, messages, footer
        ├── login.html            # Split-panel login page
        ├── register.html         # Registration with strength meter
        ├── dashboard.html        # Stats, file table, sidebar
        ├── upload.html           # Drag-drop encryption page
        ├── download.html         # Password-verify decrypt page
        ├── audit_log.html        # Security event log table
        └── confirm_delete.html   # Permanent deletion confirmation
```

Appendix D: Encryption Constants Reference

Constant	Value	Purpose
SALT_SIZE	32 bytes (256 bits)	Random salt for PBKDF2 — unique per file, prevents rainbow tables
NONCE_SIZE	12 bytes (96 bits)	Random GCM nonce — unique per encryption, prevents nonce reuse
KEY_SIZE	32 bytes (256 bits)	AES key size derived by PBKDF2 — 2^256 possible keys

PBKDF2_ITERATIONS	480,000	OWASP 2023 minimum for PBKDF2-HMAC-SHA256
VAULT_MAGIC	b'SVAULT01' (8 bytes)	Binary format identifier — verified on every decrypt
GCM_TAG_SIZE	16 bytes (128 bits)	Authentication tag appended to ciphertext by AESGCM.encrypt()
VAULT_OVERHEAD	52 bytes	Total header size: MAGIC(8) + SALT(32) + NONCE(12)